Fetal Health Classification

A beginner-friendly guide to this project: what it predicts, how to set it up, how to run the experiments, what the added feature means, and how to understand the results.

> **Important:** This is an educational machine-learning project using a public dataset. It is not a medical device and must not be used to diagnose or make decisions about a pregnancy.

1. What this project does

The project uses cardiotocography (CTG) measurements to classify each record into one of three fetal-health categories:

NSP label	Category
1	Normal
2	Suspect
3	Pathological

The notebook reads the `Raw Data` sheet in `Data/CTG.xls`, prepares model inputs, compares multiple machine-learning methods, tunes promising models, and saves results and a selected model.

The dataset has **2,126 labeled observations**. Its classes are unevenly represented:

- Normal: **1,655** (77.8%)
- Suspect: **295** (13.9%)
- Pathological: **176** (8.3%)

Because of that imbalance, accuracy by itself does not tell the whole story. This project also reports macro F1 and recall for each class.

2. The added feature

The model uses the 21 original CTG predictors and one engineered feature:

```
TotalDecelerationBurden = DL + DS + DP
```

- `DL`: light decelerations
- DS: severe decelerations
- `DP`: prolonged decelerations

The engineered value is calculated for every record from those three dataset columns. This makes **22 model inputs**. The original columns remain in the input too; the new feature does not replace them.

The feature is created in `Notebooks/fetal_health_classification.ipynb`, before the train/test split. The outcome column `NSP` and workbook metadata are not model inputs.

3. Project layout

```
Fetal_Health_Classification/
├── Data/
│   └── CTG.xls                          # Source CTG data
├── Models/
│   ├── final_model.pkl                  # Selected tuned XGBoost model
│   └── fetal_health_model.pkl           # Same selected model under an alternate
filename
├── Notebooks/
│   └── fetal_health_classification.ipynb  # Reproducible experiments
├── Reports/
│   ├── fetal_health_paper_draft.md
│   └── Fetal_Health_Research_Paper_Proper_Format.docx
├── Results/
│   ├── model_comparison.csv              # Metrics from the latest notebook run
│   └── plots/                            # Notebook-generated figures
└── README.md
```

The notebook uses relative paths such as `../data/CTG.xls` and `../results`. If your local folder names use uppercase `Data`, `Results`, and `Models`, as in the layout above, check that the notebook's paths match those names on your computer before running it.

4. Set up and run

Requirements

The notebook needs Python and the packages used in its imports, including:

- pandas, NumPy
- scikit-learn
- XGBoost and LightGBM
- imbalanced-learn
- matplotlib and seaborn

- a reader for Excel `.xls` workbooks (the notebook environment used `xlrd`)

Run the notebook

1. Open `Notebooks/fetal_health_classification.ipynb` in VS Code or Jupyter.
2. Select a Python environment that has the packages above installed.
3. Confirm the source data and notebook paths point to the correct project folders.
4. Run **all cells from top to bottom**. The later cells depend on variables and fitted models from earlier cells.
5. Check the final comparison table and `Results/model_comparison.csv` after execution.

The notebook uses a stratified 80/20 split with `random_state=42`: **1,700 records for training and 426 for holdout testing**. It uses training-only cross-validation and tuning; the holdout test set is reserved for final evaluation.

5. Experiments in the notebook

The notebook:

1. Loads and checks the CTG data and target labels.
2. Explores the class balance, feature distributions, and feature importance.
3. Evaluates nine classifier families: Logistic Regression, KNN, SVM, Decision Tree, Random Forest, Gradient Boosting, XGBoost, LightGBM, and MLP.
4. Uses stratified cross-validation on training data to compare and select models.
5. Compares Random Forest class-imbalance approaches: unweighted training, class weights, and SMOTE.
6. Compares XGBoost using all inputs with smaller top-10 and top-15 feature subsets.
7. Tunes the leading models and estimates tuned performance with nested cross-validation.
8. Saves the final comparison, plots, and selected tuned model.

SMOTE is placed inside an imbalanced-learn pipeline so oversampling is applied to training folds, not to the holdout test set.

6. What the latest measured results say

Metrics below are from the latest recorded run on the 22-feature setup. Scores are proportions: for example, `0.9084` is **90.84%**.

Selected tuned models

Model	Holdout accuracy	Holdout macro F1	Suspect recall	Nested CV macro F1 (mean ± SD)
Tuned XGBoost	95.31%	**90.84%**	81.36%	**92.74% ± 0.58 pp**

Model	Holdout accuracy	Holdout macro F1	Suspect recall	Nested CV macro F1 (mean ± SD)
Tuned Gradient Boosting	94.84%	90.11%	74.58%	91.55% ± 1.29 pp

The selected model is tuned XGBoost, selected by its nested-CV macro F1 in the current notebook workflow. On the 426-record holdout, its recalls were:

- Normal: **98.49%**
- Suspect: **81.36%**
- Pathological: **88.57%**

What changed after adding `TotalDecelerationBurden`?

The earlier 21-feature recorded results and the current 22-feature results show:

Comparison	Before (21 inputs)	After (22 inputs)	Difference
Baseline XGBoost holdout macro F1	89.68%	90.09%	**+0.41 percentage points**
Tuned XGBoost holdout macro F1	90.84%	90.84%	**No change**
Tuned XGBoost nested-CV macro F1 mean	92.40%	92.74%	**+0.33 percentage points**

So the added feature coincided with a small improvement in baseline holdout macro F1 and the tuned model's training-only nested-CV mean. It **did not improve tuned XGBoost's holdout macro F1**, and the measured increases are less than one percentage point. These are observed results from this experiment, not proof that the engineered feature alone caused a reliable improvement. In particular, the two holdout runs have the same train/test split, but model fitting and the tuning search can still affect scores; use repeated or paired cross-validation to test whether the difference is stable.

7. How to read the metrics

- **Accuracy:** fraction of all predictions that are correct. It can look high when the model mostly predicts the majority class.
- **Macro precision:** precision calculated per class and then averaged equally.

- **Macro recall:** recall calculated per class and then averaged equally.
- **Macro F1:** F1 calculated per class and then averaged equally. This is a useful overall measure for an imbalanced three-class dataset.
- **Per-class recall:** among records truly in one category, the fraction detected by the model. Suspect recall is especially useful to inspect because the categories are imbalanced.
- **Cross-validation mean ± SD:** mean and standard deviation over training folds. It is an estimate of performance consistency on this dataset, not a guarantee of future performance.
- **Percentage-point difference:** subtract the percentages directly. For example, 90.09% − 89.68% = **0.41 percentage points**, not a 0.41% relative increase.

8. Using the saved model

The saved model expects a pandas DataFrame with **22 columns and the exact names/order** stored in the model. When preparing new CTG records, calculate the derived value first:

```
import pandas as pd

# `records` must contain all 21 original predictors, including DL, DS, DP.
records = records.copy()
records["TotalDecelerationBurden"] = records[["DL", "DS", "DP"]].sum(axis=1)

feature_columns = [
    "LB", "AC", "FM", "UC", "DL", "DS", "DP", "ASTV", "MSTV", "ALTV",
    "MLTV", "Width", "Min", "Max", "Nmax", "Nzeros", "Mode", "Mean",
    "Median", "Variance", "Tendency", "TotalDecelerationBurden",
]
X_new = records.loc[:, feature_columns]
```

Then load the selected artifact and predict:

```
import pickle

with open("Models/final_model.pkl", "rb") as model_file:
    model = pickle.load(model_file)

predicted_labels = model.predict(X_new)
```

The saved XGBoost pipeline predicts encoded labels `0`, `1`, and `2`, corresponding to original `NSP` classes `1`, `2`, and `3`, respectively. Convert back to the dataset's labels with `predicted_nsp = predicted_labels + 1`. Do not include `NSP` in `X_new`.

> Predictions are for educational experimentation only. Do not use them for clinical decisions.

9. Reproducibility and limitations

- The data split and many estimators use `random_state=42`, but exact output may vary with package versions and execution environment.
- Only one labeled dataset and one holdout split are represented here. There is no external validation.
- The classes are imbalanced, and errors between classes may have different costs.
- A small score increase on one run does not establish a real-world improvement. Prefer repeated, paired stratified cross-validation, report uncertainty, and preserve an untouched test set.
- Feature importance and predictive associations are not evidence of causation.
- This model is not clinically validated and is not intended to diagnose fetal health.

10. Main files

- Experiment workflow: `Notebooks/fetal_health_classification.ipynb`
- Current metric table: `Results/model_comparison.csv`
- Selected model: `Models/final_model.pkl`
- Alternate model filename: `Models/fetal_health_model.pkl`
- Research draft: `Reports/fetal_health_paper_draft.md`
- Formatted paper: `Reports/Fetal_Health_Research_Paper_Proper_Format.docx`